Examen de Fin de Module Régional N°204
Direction Régionale Casablanca Settat
Développement Front-end.
Année 2022/2023

Durée: 2H30 *Barème: /40*

Exercice 1 : (ReactJS)

NB: les parties 1 et 2 sont indépendantes

Partie 1:

1. On suppose que NodeJs est déjà installé sur votre machine

   Donner les lignes de commande pour créer cette application qui portera le nom suivant 2 EFM-votre Nom et pour la lancer

2. Donner la signification de chaque ordre suivant:

```
const [nb, setNb] = useState(0)

     useEffect(() => setNb(Stagiaires.length))

     useEffect(() => setNb(Stagiaires.length), [])

     useEffect(() => setNb(Stagiaires.length), [a, b])
```

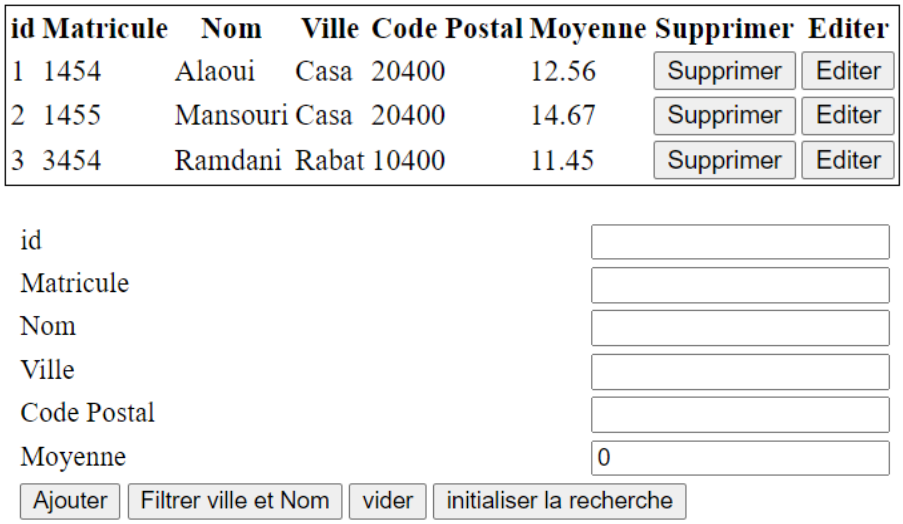
Partie 2 :

On souhaite développer une application en **ReactJS**

On représente une classe de stagiaires sous forme d'un tableau ayant la structure suivante:

```
Stagiaires = [
  {
    id: 1,
    matricule: 1454,
    nom: "Alaoui",
    codepostal: 20400,
    ville: "Casa",
    moyenne: 14.67
  },
  {
    id: 2,
    matricule: 1455,
    nom: "Mansouri",
    codepostal: 20400,
    ville: "Casa",
    moyenne: 14.67
  },
  . . . ]
```

On souhaite créer un composant fonctionnel App dans le fichier App.js pour gérer toutes les fonctionnalités de l'interface ayant l'aperçu suivant :



- La Moyenne générale la plus élevée : 14.6
- La Moyenne générale la moins élevée : 11.45
- Le Moyenne de la classe : 12.333333333333334

Tableau de recherche Vide

1. Écrire le code nécessaire pour **Ajouter** un nouveau stagiaire selon le respect des contraintes suivantes : l'id du stagiaire est généré automatiquement, la moyenne générale du stagiaire doit être comprise entre 0 et 20 et tous les champs sont obligatoires à saisir, l'id doit être non modifiable, Le matricule doit être unique.
2. Écrire le code pour afficher la liste des stagiaires au démarrage de l'application, prévoir l'écriture du message tableau des stagiaires vide en rouge si la liste est vide.
3. Écrire le code du bouton Supprimer permettant de supprimer le stagiaire concerné par Faction de suppression.
4. Écrire le code du bouton Editer, le clic sur le bouton permet d'afficher dans les zones texte les informations correspondantes du stagiaire concerne par l'action.
5. Écrire le code du bouton vider, le click permet de vider toutes les zones de texte sauf l'id
6. Écrire le code du bouton Filtrer ville et nom qui permet de filtrer le tableau des stagiaires selon la valeur du nom et de la ville saisies dans les zones de texte, le résultat est stocké dans un tableau d'objets appelé stagiaires2 et l'afficher sous forme d'une liste HTML non ordonnée.

   Prévoir l'écriture du message **« Tableau de recherche vide »** au cas où le tableau stagiaires2 est vide.

7. Écrire le code du bouton initialiser recherche pour vider le tableau stagiaire2 à la demande.
8. Afficher, dans une liste non ordonnée, la moyenne générale la plus élevée, la moins élevée et la moyenne générale de la classe.

Excercice 2 : (Redux)

Composant : App

Composant : Cinputs

valeur1 12

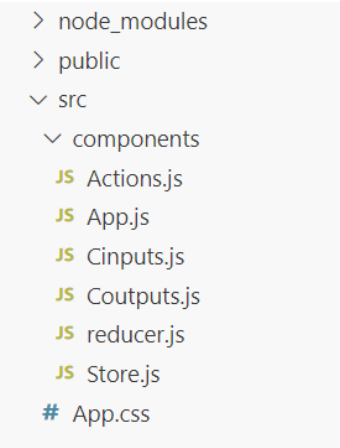
valeur2 3

\+ - * /

Composants : Coutputs.js

Resititution de l'operation du store est comme suit:
12-3=9
le nb operation: 8

L'objectif de cet exercice est de réaliser une calculette basique (voir interface ci-dessus) en utilisant **Redux** le projet ayant l'arborescence suivante :



NB : vous pouvez proposer une autre arborescence autre que celle proposée.

On suppose que le code du fichier App.js est comme suit :

```
import React from 'react';
import Cinputs from './Cinputs';
import Coutputs from './Coutputs';
export default class App() {
    return (
        <>
            <Cinputs />
            <br />
            <Coutputs />
        </>
    );
}
```

Travail à faire :

1. Donner la ligne de commande à écrire dans le terminal pour installer Redux dans une application React supposée déjà créer.
2. Écrire le code du fichier **Store.js** qui contient l'ordre de création de l'objet store, on suppose que cet objet est bien configuré dans le fichier index.js comme suit :

   ```
   Root.render(<><Provider store={store}><App/></Provider></>)
   ```

3. Écrire le code du fichier **Actions.js** contenant les fonctions appelées pour chaque action nécessaire pour ce cas à savoir : l'addition, la soustraction, la multiplication et la division.
4. Écrire le code du fichier **reducer.js** contenant la fonction `reducer(state, action)`.
5. Écrire le code du fichier Cinputs.js pour créer le composant fonctionnel **Cinputs** qui retourne: quatre boutons: pour effectuer les opérations: Addition, Multiplication, Soustraction et Division ainsi que deux zones de texte pour la saisie des valeurs, prévoir la connexion avec le store pour l'appel des fonctions définies dans le fichier **Actions.js**.

   (Vous pouvez utiliser **`useDispatch()`** de la bibliothèque **react-redux** (ou une autre technique))
6. Écrire le code du fichier **Coutputs.js** pour créer le composant fonctionnel **Coutputs** afin d'afficher tous les membres de l'opération effectuée dans le composant **Cinputs** le résultat doit être affichée dans le composant **Coutputs** sous le format: valeur1 opération valeur2 = résultat

   (prévoir l'utilisation du **`useSelector()`** de la bibliothèque **react-redux** (ou une autre technique))